INTERNAL DOCUMENT · DEVELOPER ONBOARDING

아임딜러
신규 개발자 온보딩 가이드

이 문서는 아임딜러 프로젝트에 합류한 개발자가
프로젝트의 **왜, 무엇을, 어떻게**를 빠르게 이해하기 위한 자료입니다.
코드를 보기 전에 반드시 먼저 읽을 것.

작성일	현재 단계	기술 스택
2026-04-07	1단계 (시장 진입)	Next.js 14 · TypeScript · Supabase

목차 · CONTENTS

01	이 서비스가 뭔가요?	서비스 철학 · 존재 이유
02	3단계 로드맵	현재 어디까지 왔는가
03	기술 스택	왜 이 기술을 쓰는가
04	프로젝트 구조	폴더 트리 · 파일 역할
05	페이지별 역할	고객용 · 관리자용
06	핵심 비즈니스 로직	AI 추천 엔진 · 견적 계산 · DB 관계
07	API 엔드포인트	구현된 것 · 예정인 것
08	디자인 시스템	컬러 · 원칙 · UI 규칙
09	환경 설정 · 로컬 실행	시작하는 방법
10	현재 개발 상태	완료 · 진행 중 · 남은 것
11	참고 문서	어디서 더 찾아볼까
12	자주 헷갈리는 것들	Q & A

01 이 서비스가 뭔가요?

아임딜러는 자동차 장기렌트·리스 견적 플랫폼이다.
그런데 단순 견적 사이트가 아니다. **기존 시장의 구조적 문제를 뒤집는 것**이 핵심이다.

기존 시장의 문제

상담 DB 수집이 진짜 목적

이름·전화번호 입력해야 견적 공개.
견적이 미끼, 영업전화가 진짜 목적.
고객은 비교하는 게 아니라 영업당하는 느낌.

아임딜러가 뒤집는 것

고객이 먼저 탐색하고 신뢰

개인정보 없이도 견적 확인 가능.
AI가 차량 추천 + 이유 설명.
상담은 그 이후 자연스럽게 연결.

> ❌ 절대 원칙: "상담접수형 구조로 보이면 실패다" — 이름/전화번호 먼저 요구하는 UI는 절대 금지

4가지 핵심 방향

1. 견적을 미끼로 상담 유도하는 방식이 아닌, **고객이 먼저 판단할 수 있는 구조**
2. AI가 전면에 보이되 과장되지 않고 **실제 도움을 주는 구조**
3. 계약 전환 이후에도 **고객과 관계를 유지하는 구조**
4. 내부에서 검증한 운영 체계를 바탕으로 **외부 SaaS 확장까지 고려한 구조**

02 3단계 로드맵

현재 개발 중

1단계

시장 진입

프론트 경험, AI 인상, 신뢰 구조,
빠른 베타 오픈

2단계

고객 리텐션

계약 후 관계 유지
보험·사고·운행일지·담당자 연결

3단계

플랫폼 확장

소형 에이전시 대상
SaaS CRM 멀티테넌트 확장

> 현재는 1단계만 개발 중이다. 단, 2·3단계 확장을 고려한 구조로 설계되어 있다.
> 예: `sessionId → userId` 마이그레이션, `tenantId` 컬럼 추가 구조, Row Level Security 준비

03 기술 스택

항목	기술	선택 이유
프레임워크	Next.js 14 (App Router)	SSR + API Routes 통합, 단일 레포로 프론트·백 관리
언어	TypeScript	전체 코드베이스, 타입 안전성
스타일링	Tailwind CSS	커스텀 디자인 토큰 포함, 일관성 유지
DB	PostgreSQL (Supabase)	관리형 DB, Row Level Security 내장
ORM	Prisma	타입 안전한 DB 접근, 마이그레이션 관리
아이콘	Lucide React	일관된 아이콘 세트
유틸	clsx, tailwind-merge	조건부 클래스 관리
검증	Zod	API Route 입력값 런타임 검증

04 프로젝트 구조

```
src/
├── app/
│   ├── (public)/          # 고객용 페이지 그룹
│   │   ├── layout.tsx     # 공통 Header 포함
│   │   ├── page.tsx       # 홈 (/)
│   │   ├── recommend/     # AI 추천 (/recommend)
│   │   ├── cars/          # 차량 탐색 (/cars, /cars/[slug])
│   │   ├── quote/         # 견적 계산기 (/quote)
│   │   └── about/         # 브랜드 소개 (/about)
│   │
│   ├── (admin)/           # 관리자 페이지 그룹
│   │   ├── layout.tsx     # 사이드바 포함
│   │   └── admin/         # (/admin, /admin/vehicles, ...)
│   │
│   └── api/               # API Routes
│       ├── vehicles/      # 차량 API
│       ├── recommend/     # AI 추천 API
│       └── logs/          # 탐색 로그 API
│
├── components/
│   ├── ui/                # 디자인 시스템 원자 컴포넌트
│   ├── layout/            # Header, BottomNav
│   ├── home/              # 홈 섹션 컴포넌트
│   ├── recommend/         # AI 추천 플로우
│   ├── cars/              # 차량 관련
│   ├── quote/             # 견적 관련
│   └── admin/             # 관리자 컴포넌트
│
├── lib/
│   ├── prisma.ts          # Prisma 클라이언트 싱글톤
│   ├── ai-recommender.ts  # ★ AI 추천 엔진 (핵심)
│   ├── quote-calculator.ts # ★ 견적 계산 로직 (순수 함수)
│   └── use-tracking.ts    # 탐색 로그 훅
│
├── types/                 # TypeScript 타입 정의
├── constants/             # 디자인 토큰, mock 데이터, 기본값
└── prisma/
    ├── schema.prisma      # DB 스키마 전체 모델 정의
    ├── migrations/        # 마이그레이션 이력
    └── seed.ts            # 초기 데이터 스크립트
```

05 페이지별 역할

고객용 (Public)

경로	역할	핵심 포인트
`/`	홈	AI 추천 진입 유도, 브랜드 첫인상, 신뢰 요소 노출
`/recommend`	AI 추천	업종→목적→예산→성향 4단계 입력 후 추천 결과
`/cars`	차량 탐색	전체 차량 목록, 카테고리·브랜드 필터
`/cars/[slug]`	차량 상세	트림·옵션 선택, 견적 미리보기
`/quote`	견적 계산기	조건 직접 입력 → 3가지 시나리오 견적
`/about`	브랜드 소개	아임딜러 철학, 신뢰 요소

관리자용 (Admin)

경로	역할
`/admin`	대시보드 — 주요 지표 요약
`/admin/vehicles`	차량 등록·수정·노출 관리
`/admin/vehicles/[id]/trims`	트림·옵션 관리
`/admin/quote-variables`	금리·잔존가치 등 견적 운영값
`/admin/recommendations`	AI 추천 기초 데이터 (스코어 행렬)
`/admin/content`	메인 배너·프로모션·공지
`/admin/logs`	탐색 흐름·추천 클릭 로그 조회
`/admin/notes`	운영 메모

06 핵심 비즈니스 로직

6-1. AI 추천 엔진 `src/lib/ai-recommender.ts`

외부 AI API 없이 **규칙 기반 스코어링**으로 구현되어 있다.

```
# 동작 흐름
1. DB에서 노출 가능한 차량 전체 조회
2. 차량의 vehicleCode로 금융사별 RateConfig 조회
3. 각 차량의 예산 적합도 계산 (월 납입금 추정)
4. 업종·목적별 스코어 행렬(RecommendationConfig)로 추가 점수 부여
5. 종합 점수 상위 3개 차량 추천
6. 추천된 차량에 대해 3가지 시나리오(보수형/표준형/공격형) 계산
```

> 향후 방향: 탐색 로그 데이터 축적 → 2단계에서 ML 기반으로 고도화 예정. 1단계부터 로그 구조를 설계해둔 이유가 이것.

6-2. 견적 계산 로직 `src/lib/quote-calculator.ts`

순수 함수로 구현되어 있다 (DB 의존 없음, 테스트 용이).

```
# 월 납입금 공식
월 납입금 = (차량가 × (1 − 잔존가치율) × (1 − 선납금비율) × 금리계수)
           + (수수료율 × 차량가 / 계약기간)
           + (세금 + 부대비용 / 계약기간)
           − 프로모션할인 / 계약기간
```

시나리오	특징	UI 처리
보수형	낮은 약정거리, 높은 보증금 → 월 납입 낮음, 안전한 선택	일반
표준형	균형 조건	**기본 추천 (강조 표시)**
공격형	높은 약정거리, 낮은 보증금 → 월 납입 가장 낮음	일반

6-3. 데이터 모델 핵심 관계

```
Vehicle (차량)
  └── Trim[] (트림)
        └── TrimOption[] (옵션)
        └── QuoteVariable[] (견적변수 ← 금리·잔존가치 등, 운영자가 관리)

FinanceCompany (금융사)
  └── RateConfig[] (회수율 매트릭스 ← 금융사×차명 단위)

RecommendationConfig (AI 추천 기초 데이터 ← 운영자 관리)
  └── scoreMatrix: { "업종": { "목적": 점수 } }
```

```
RecommendationLog (추천 로그 ← AI 고도화용 데이터 축적)
ExplorationLog    (탐색 로그 ← 이탈 지점 파악용)
```

07 API 엔드포인트

공개 API (구현 완료)

Method	경로	설명
GET	/api/vehicles	차량 목록 (category, brand, sort, page 필터)
GET	/api/vehicles/[slug]	차량 상세 + 트림 목록
GET	/api/vehicles/[slug]/quote	차량 기반 견적 계산
POST	/api/recommend	AI 추천 (업종·목적·예산·성향 입력)
GET	/api/recommend/[sessionId]	추천 결과 재조회
POST	/api/logs/exploration	탐색 흐름 기록
POST	/api/logs/recommendation-click	추천 차량 클릭 기록
POST	/api/logs/quote-view	견적 결과 조회 기록

관리자 API (구현 예정)

Method	경로	설명
CRUD	/api/admin/cars	차량 관리 — 인증 필수
CRUD	/api/admin/cars/[id]/trims	트림 관리 — 인증 필수
GET/PUT	/api/admin/quote-variables	견적 변수 조회·수정
CRUD	/api/admin/content	배너·콘텐츠 관리
GET	/api/admin/logs	탐색·추천 로그 조회

08 디자인 시스템

상세 내용은 `DESIGN.md`에 전체 정의되어 있다. 핵심만 요약.

컬러 팔레트

역할	색상	용도
`Primary`	`#000666` Deep Navy	CTA, 활성 상태, 핵심 UI 지배
`Secondary`	`#71749A` Slate Blue	보조 텍스트, 비활성 상태
`Tertiary`	`#5C1800` Deep Burgundy	파괴적 상태, 3차 액션에만 제한 사용
`Neutral`	`#F8F9FA` Off White	페이지 배경

레이아웃 원칙

항목	값
개발 우선순위	Desktop-First (1280px 기준) — 모바일은 `max-md:` 프리픽스로 최소 대응
`Max content width`	1200px
카드 `radius`	12px
버튼 `radius`	8px
`Base unit`	8px

UI 절대 규칙

- 상담접수 유도 버튼·문구 **절대 금지**
- **"월 xx원부터"** 같은 최저가 단순 강조 금지
- 신뢰 배지(*허위견적 없음 · 개인정보 없이 견적 확인 · 상담 압박 없음*)는 화면 내 자연스럽게 배치
- 텍스트는 순수 검정(`#000000`) 대신 `#333333` 사용

09 환경 설정 · 로컬 실행

필요한 환경변수 `.env.local`

```
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_...
SUPABASE_SERVICE_ROLE_KEY=sb_secret_...
DATABASE_URL=postgresql://...pooler.supabase.com:6543/postgres
DIRECT_URL=postgresql://...supabase.com:5432/postgres
```

로컬 실행 순서

```
# 1. 의존성 설치
npm install

# 2. DB 마이그레이션 (이미 적용됨, 처음 설정 시만)
npx prisma migrate deploy

# 3. 초기 데이터 입력 (차량/트림/견적변수 등)
npx prisma db seed

# 4. 개발 서버 실행
npm run dev
```

> DB 데이터가 없으면 차량 목록, AI 추천 등 핵심 기능이 빈 화면으로 보인다. `prisma db seed` 를 반드시 실행할 것.

10 현재 개발 상태

- ✅ 전체 공개 페이지 UI (홈, 추천, 차량, 견적, 소개)
- ✅ AI 추천 4단계 플로우
- ✅ 견적 계산 로직 (3가지 시나리오)
- ✅ 공개 API Routes
- ✅ Prisma 스키마 + DB 마이그레이션 완료
- ✅ 탐색 로그 수집 구조
- ❌ **관리자 인증** — 현재 /admin이 인증 없이 누구나 접근 가능
- ❌ **Admin API Routes** — 디렉토리만 존재, 파일 0개
- ❌ **관리자 페이지** — 대시보드·차량목록만 존재 (mock 데이터)
- ❌ **DB 실제 데이터** — 차량/트림/견적변수 데이터 없음
- 🔶 Quote 저장/공유 — /api/quote/save, /quote/[id] 미구현
- 🔶 API 경로 불일치 — 스펙은 /api/cars지만 현재 /api/vehicles

작업 우선순위

PHASE 1 · 배포 최소 조건

1. DB 시드 데이터 입력
2. API 경로 통일 (/api/cars)
3. 관리자 인증 구현
4. Admin API Routes 구현

PHASE 2 · 완성도

5. 나머지 관리자 페이지
6. Quote 저장/공유 기능
7. Footer 컴포넌트
8. /api/cars/popular 엔드포인트

상세 내용은 `PRODUCTION_CHECKLIST.md` 참고

11 참고 문서

문서	내용
`imdealers-RFP.md`	서비스 철학, 3단계 로드맵, 핵심 원칙 — 가장 먼저 읽을 것
`PROJECT_SPEC.md`	페이지 목록, 컴포넌트 구조, DB 스키마, API 목록
`DESIGN.md`	디자인 시스템 전체 (컬러, 타이포, 컴포넌트 스펙)
`PRODUCTION_CHECKLIST.md`	배포 전 미완성 항목 체크리스트
`prisma/schema.prisma`	DB 모델 전체 정의

12 자주 헷갈리는 것들

Q. AI 추천이 ChatGPT 같은 외부 AI를 쓰나요?

아니다. 외부 AI API 없이 규칙 기반 스코어링으로 구현되어 있다. 운영자가 `RecommendationConfig`에 업종·목적별 점수 행렬을 입력하면 그것을 기반으로 추천한다. `src/lib/ai-recommender.ts` 참고.

Q. 로그인이나 인증은 어떻게 되어 있나요?

현재 고객용 페이지는 인증이 없다 (비회원 세션 UUID 방식). 관리자 인증은 아직 미구현 상태다. `/admin`이 현재 누구나 접근 가능하니 주의할 것.

Q. 차량 데이터는 어디서 관리하나요?

관리자 페이지(`/admin`)에서 직접 등록한다. 현재는 `constants/mock-vehicles.ts`의 mock 데이터를 쓰고 있고, 관리자 기능 구현 후 실제 DB 데이터로 전환 예정이다.

Q. 견적이 실제 계약 가능한 금액인가요?

운영자가 `QuoteVariable`(금리·잔존가치율·수수료율 등)을 관리자 페이지에서 직접 입력하고, 그 값으로 계산한다. 유인용 최저가 방식이 아닌 실제 운영 가능한 견적이 원칙이다.

Q. 채널톡은 뭔가요?

고객 상담 채널이다. 견적 확인 후 "상담 요청" 클릭 시 채널톡으로 연결된다. 이름·전화번호를 강요하지 않고 고객이 준비됐을 때 먼저 연락할 수 있는 구조로 설계되어 있다. `src/components/quote/ChannelTalkButton.tsx` 참고.

Q. Tailwind 클래스를 수정해도 되나요?

가능하지만, `DESIGN.md`의 디자인 시스템을 먼저 확인해야 한다. 커스텀 토큰(`primary`, `secondary`, `tertiary` 등)은 `tailwind.config.ts`에 정의되어 있다. 임의 색상 하드코딩은 지양할 것.